Caffe2 Quick Start Guide

Modular and scalable deep learning made easy

Ashwin Nanjappa

Packt
www.packt.com

Caffe2 Quick Start Guide

Modular and scalable deep learning made easy

Ashwin Nanjappa

Packt

BIRMINGHAM - MUMBAI

Caffe2 Quick Start Guide

Commissioning Editor: Amey Varangaonkar
Acquisition Editor: Siddharth Mandal
Content Development Editor: Mohammed Yusuf Imaratwale
Technical Editor: Rutuja Vaze
Copy Editor: Safis Editing
Project Coordinator: Kinjal Bari
Proofreader: Safis Editing
Indexer: Pratik Shirodkar
Graphics: Jason Monteiro
Production Coordinator: Jyoti Chauhan

First published: May 2019

Production reference: 1310519

Published by Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham
B3 2PB, UK.

ISBN 978-1-78913-775-0

www.packtpub.com

mapt.io

Mapt is an online digital library that gives you full access to over 5,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Mapt is fully searchable
- Copy and paste, print, and bookmark content

Packt.com

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at www.packt.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at customercare@packtpub.com for more details.

At www.packt.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Contributors

About the author

Ashwin Nanjappa is a senior architect at NVIDIA, working in the TensorRT team on improving deep learning inference on GPU accelerators. He has a PhD from the National University of Singapore in developing GPU algorithms for the fundamental computational geometry problem of 3D Delaunay triangulation. As a post-doctoral research fellow at the BioInformatics Institute (Singapore), he developed GPU-accelerated machine learning algorithms for pose estimation using depth cameras. As an algorithms research engineer at Visenze (Singapore), he implemented computer vision algorithm pipelines in C++, developed a training framework built upon Caffe in Python, and trained deep learning models for some of the world's most popular online shopping portals.

About the reviewers

Gianni Rosa Gallina is an Italian senior software engineer and architect who has been focused on emerging technologies, AI, and virtual/augmented reality since 2013. Currently, he works in Deltatre's Innovation Lab, prototyping solutions for next-generation sport experiences and business services. Besides that, he has 10+ years of certified experience as consultant on Microsoft and .NET technologies (including technologies such as the Internet of Things, the cloud, and desktop/mobile apps). Since 2011, he has been awarded Microsoft MVP in the Windows Development category. He has been a Pluralsight Author since 2013 and is a speaker at national and international conferences.

Ryan Riley has been in the futures and derivatives industry for almost 20 years. He received bachelor's and master's degrees from DePaul University in applied statistics. Doing his coursework in math meant he had to teach himself how to program, thus forcing him to read more technical books on programming than someone would otherwise. Ryan has worked with numerous AI libraries in various languages and is currently using the Caffe2 C++ library to develop and implement futures and derivatives trading strategies at PNT Financial.

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit `authors.packtpub.com` and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Table of Contents

Preface

Caffe2 is a popular deep learning framework designed with a focus on scalability, high performance, and portability. Written in C++, it has both a C++ API and a Python API. This book is a guide for you to quickly get started with Caffe2. It will cover the topics of installing Caffe2, composing networks using its operators, training models, and deploying models to inference engines, devices at the edge, and the cloud. It will also show you how to work with Caffe2 and other deep learning frameworks using the ONNX interchange format.

Who this book is for

Data scientists and machine learning engineers who wish to create fast and scalable deep learning models in Caffe2 will find this book to be very useful.

What this book covers

`Chapter 1`, *Introduction and Installation*, introduces Caffe2 and examines how to build and install it.

`Chapter 2`, *Composing Networks*, teaches you about Caffe2 operators and how to compose them to build a simple computation graph and a neural network to recognize handwritten digits.

`Chapter 3`, *Training Networks*, gets into how to use Caffe2 to compose a network for training and how to train a network to solve the MNIST problem.

`Chapter 4`, *Working with Caffe*, explores the relationship between Caffe and Caffe2 and how to work with models trained in Caffe.

`Chapter 5`, *Working with Other Frameworks*, looks at contemporary deep learning frameworks such as TensorFlow and PyTorch and how we can exchange models from and to Caffe2 and these other frameworks.

`Chapter 6`, *Deploying Models to Accelerators for Inference*, talks about inference engines and how they are an essential tool for the final deployment of a trained Caffe2 model on accelerators. We focus on two types of popular accelerators: NVIDIA GPUs and Intel CPUs. We look at how to install and use TensorRT for deploying our Caffe2 model on NVIDIA GPUs. We also look at the installation and use of OpenVINO for deploying our Caffe2 model on Intel CPUs and accelerators.

`Chapter 7`, *Caffe2 at the Edge and in the cloud*, covers two applications of Caffe2 to demonstrate its ability to scale. As an application of Caffe2 with edge devices, we look at how to build Caffe2 on Raspberry Pi single-board computers and how to run Caffe2 applications on them. As an application of Caffe2 with the cloud, we look at the use of Caffe2 in Docker containers.

To get the most out of this book

Some understanding of basic machine learning concepts and prior exposure to programming languages such as C++ and Python will be useful.

Download the example code files

You can download the example code files for this book from your account at `www.packt.com`. If you purchased this book elsewhere, you can visit `www.packt.com/support` and register to have the files emailed directly to you.

You can download the code files by following these steps:

1. Log in or register at `www.packt.com`.
2. Select the **SUPPORT** tab.
3. Click on **Code Downloads & Errata**.
4. Enter the name of the book in the **Search** box and follow the onscreen instructions.

Once the file is downloaded, please make sure that you unzip or extract the folder using the latest version of:

- WinRAR/7-Zip for Windows
- Zipeg/iZip/UnRarX for Mac
- 7-Zip/PeaZip for Linux

The code bundle for the book is also hosted on GitHub at `https://github.com/PacktPublishing/Caffe2-Quick-Start-Guide`. In case there's an update to the code, it will be updated on the existing GitHub repository.

We also have other code bundles from our rich catalog of books and videos available at `https://github.com/PacktPublishing/`. Check them out!

Conventions used

There are a number of text conventions used throughout this book.

`CodeInText`: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. Here is an example: "The output values of a `SoftMax` function have nice properties."

A block of code is set as follows:

```
model = model_helper.ModelHelper("MatMul model")
model.net.MatMul(["A", "B"], "C")
```

Any command-line input or output is written as follows:

```
$ sudo apt-get update
```

Bold: Indicates a new term, an important word, or words that you see onscreen. For example, words in menus or dialog boxes appear in the text like this. Here is an example: " For example, for Ubuntu, it gives me the option of downloading a **Customizable Package** or a single large **Full Package**."

Warnings or important notes appear like this.

Tips and tricks appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book, mention the book title in the subject of your message and email us at `customercare@packtpub.com`.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you would report this to us. Please visit `www.packt.com/submit-errata`, selecting your book, clicking on the Errata Submission Form link, and entering the details.

Piracy: If you come across any illegal copies of our works in any form on the Internet, we would be grateful if you would provide us with the location address or website name. Please contact us at `copyright@packt.com` with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit `authors.packtpub.com`.

Reviews

Please leave a review. Once you have read and used this book, why not leave a review on the site that you purchased it from? Potential readers can then see and use your unbiased opinion to make purchase decisions, we at Packt can understand what you think about our products, and our authors can see your feedback on their book. Thank you!

For more information about Packt, please visit `packt.com`.

1
Introduction and Installation

Welcome to the Caffe2 Quick Start Guide. This book aims to provide you with a quick introduction to the Caffe2 deep learning framework and how to use it for training and deployment of deep learning models. This book uses code samples to create, train, and run inference on actual deep learning models that solve real problems. In this way, its code can be applied quickly by readers to their own applications.

This chapter provides a brief introduction to Caffe2 and shows you how to build and install it on your computer. In this chapter, we will cover the following topics:

- Introduction to deep learning and Caffe2
- Building and installing Caffe2
- Testing Caffe2 Python API
- Testing Caffe2 C++ API

Introduction to deep learning

Terms such as **artificial intelligence** (**AI**), **machine learning** (**ML**), and **deep learning (DL)** are popular right now. This popularity can be attributed to significant improvements that deep learning techniques have brought about in the last few years in enabling computers to see, hear, read, and create. First and foremost, we'll introduce these three fields and how they intersect:

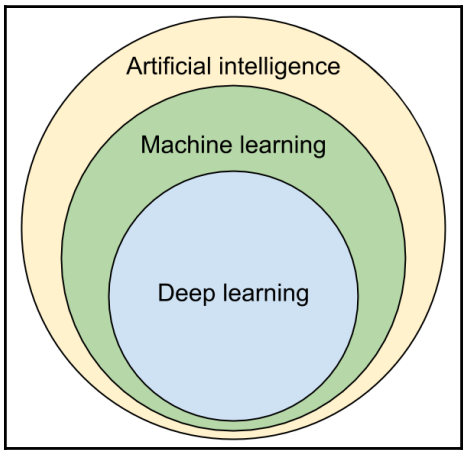

Figure 1.1: Relationship between deep learning, ML, and AI

AI

Artificial intelligence (**AI**) is a general term used to refer to the intelligence of computers, specifically their ability to reason, sense, perceive, and respond. It is used to refer to any non-biological system that has intelligence, and this intelligence is a consequence of a set of rules. It does not matter in AI if those sets of rules were created manually by a human, or if those rules were automatically learned by a computer by analyzing data. Research into AI started in 1956, and it has been through many ups and a couple of downs, called **AI winters**, since then.

ML

Machine learning (**ML**) is a subset of AI that uses statistics, data, and learning algorithms to teach computers to learn from given data. This data, called **training data**, is specific to the problem being solved, and contains examples of input and the expected output for each input. ML algorithms learn models or representations automatically from training data, and these models can be used to obtain predictions for new input data.

There are many popular types of models in ML, including **artificial neural networks** (**ANNs**), Bayesian networks, **support vector machines** (**SVM**), and random forests. The ML model that is of interest to us in this book is ANN. The structure of ANNs are inspired by the connections in the brain. These neural network models were initially popular in ML, but later fell out of favor since they required enormous computing power that was not available at that time.

Deep learning

Over the last decade, utilization of the parallel processing capability of **graphics processing units** (**GPUs**) to solve general computation problems became popular. This type of computation came to be known as **general-purpose computing on GPU (GPGPU)**. GPUs were quite affordable and were easy to use as accelerators by using GPGPU programming models and APIs such as **Compute Unified Device Architecture** (**CUDA**) and **Open Computing Language** (**OpenCL**). Starting in 2012, neural network researchers harnessed GPUs to train neural networks with a large number of layers and started to generate breakthroughs in solving computer vision, speech recognition, and other problems. The use of such deep neural networks with a large number of layers of neurons gave rise to the term **deep learning**. Deep learning algorithms form a subset of ML and use multiple layers of abstraction to learn and parameterize multi-layer neural network models of data.

Introduction to Caffe2

The popularity and success of deep learning has been motivated by the creation of many popular and open source deep learning frameworks that can be used for training and inference of neural networks. **Caffe** was one of the first popular deep learning frameworks. It was created by *Yangqing Jia* at UC Berkeley for his PhD thesis and released to the public at the end of 2013. It was primarily written in C++ and provided a C++ API. Caffe also provided a rudimentary Python API wrapped around the C++ API. The Caffe framework created networks using layers. Users created networks by listing down and describing its layers in a text file commonly referred to as a **prototxt**.

Following the popularity of Caffe, universities, corporations, and individuals created and launched many deep learning frameworks. Some of the popular ones today are Caffe2, TensorFlow, MXNet, and PyTorch. TensorFlow is driven by Google, MXNet has the support of Amazon, and PyTorch was primarily developed by Facebook.

Caffe's creator, Yangqing Jia, moved to Facebook, where he created a follow-up to Caffe called Caffe2. Compared to the other deep learning frameworks, Caffe2 was designed to focus on scalability, high performance, and portability. Written in C++, it has both a C++ API and a Python API.

Caffe2 and PyTorch

Caffe2 and PyTorch are both popular DL frameworks, maintained and driven by Facebook. PyTorch originates from the **Torch** DL framework. It is characterized by a Python API that is easy for designing different network structures and experimenting with training parameters and regimens on them. While PyTorch could be used for inference in production applications on the cloud and in the edge, it is not as efficient when it comes to this.

Caffe2 has a Python API and a C++ API. It is designed for practitioners who tinker with existing network structures and use pre-trained models from PyTorch, Caffe, and other DL frameworks, and ready them for deployment inside applications, local workstations, low-power devices at the edge, mobile devices, and in the cloud.

Having observed the complementary features of PyTorch and Caffe2, Facebook has a plan to merge the two projects. As we will see later, Caffe2 source code is already organized as a subdirectory under the PyTorch Git repository. In the future, expect more intermingling of these two projects, with a final goal of fusing the two together to create a single DL framework that is easy to experiment with and tinker, efficient to train and deploy, and that can scale from the cloud to the edge, from general-purpose processors to special-purpose accelerators.

Hardware requirements

Working with deep learning models, especially the training process, requires a lot of computing power. While you could train a popular neural network on the CPU, it could typically take many hours or days, depending on the complexity of the network. Using GPUs for training is highly recommended since they typically reduce the training time by an order of magnitude or more compared to CPUs. Caffe2 uses CUDA to access the parallel processing capabilities of NVIDIA GPUs. CUDA is an API that enables developers to use the parallel computation capabilities of an NVIDIA GPU, so you will need to use an NVIDIA GPU. You can either install an NVIDIA GPU on your local computer, or use a cloud service provider such as Amazon AWS that provides instances with NVIDIA GPUs. Please take note of the running costs of such cloud instances before you use them for extended periods of training.

Once you have trained a model using Caffe2, you can use CPUs, GPUs, or many other processors for inference. We will explore a few such options in `Chapter 6`, *Deploying Models to Accelerators for Inference*, and `Chapter 7`, *Caffe2 at the Edge and in the cloud*, later in the book.

Software requirements

A major portion of deep learning research and development is currently taking place on Linux computers. **Ubuntu** is a distribution of Linux that happens to be very popular for deep learning research and development. We will be using Ubuntu as the operating system of choice in this book. If you are using a different flavor of Linux, you should be able to search online for commands similar to Ubuntu commands for most of the operations described here. If you use Windows or macOS, you will need to replace the Linux commands in this book with equivalent commands. All the code samples should work on Linux, Windows, and macOS with zero or minimal changes.

Building and installing Caffe2

Caffe2 can be built and installed from source code quite easily. Installing Caffe2 from source gives us more flexibility and control over our application setup. The build and install process has four stages:

1. Installing dependencies
2. Installing acceleration libraries

3. Building Caffe2
4. Installing Caffe2

Installing dependencies

We first need to install packages that Caffe2 is dependent on, as well as the tools and libraries required to build it.

1. First, obtain information about the newest versions of Ubuntu packages by querying their online repositories using the `apt-get` tool:

```
$ sudo apt-get update
```

2. Next, using the `apt-get` tool, install the libraries that are required to build Caffe2, and that Caffe2 requires for its operation:

```
$ sudo apt-get install -y --no-install-recommends \
      build-essential \
      cmake \
      git \
      libgflags2 \
      libgoogle-glog-dev \
      libgtest-dev \
      libiomp-dev \
      libleveldb-dev \
      liblmdb-dev \
      libopencv-dev \
      libopenmpi-dev \
      libsnappy-dev \
      libprotobuf-dev \
      openmpi-bin \
      openmpi-doc \
      protobuf-compiler \
      python-dev \
      python-pip
```

These packages include tools required to download Caffe2 source code (Git) and to build Caffe2 (`build-essential`, `cmake`, and `python-dev`). The rest are libraries that Caffe2 is dependent on, including Google Flags (`libgflags2`), Google Log (`libgoogle-glog-dev`), Google Test (`libgtest-dev`), LevelDB (`libleveldb-dev`), LMDB (`liblmdb-dev`), OpenCV (`libopencv-dev`), OpenMP (`libiomp-dev`), OpenMPI (`openmpi-bin and openmpi-doc`), Protobuf (`libprotobuf-dev and protobuf-compiler`), and Snappy (`libsnappy-dev`).

3. Finally, install the Python Pip tool and use it to install other Python libraries such as `NumPy` and `Protobuf` Python APIs that are useful when working with Python:

```
$ sudo apt-get install -y --no-install-recommends python-pip

$ pip install --user \
      future \
      numpy \
      protobuf
```

Installing acceleration libraries

Using Caffe2 to train DL networks and using them for inference involves a lot of math computation. Using acceleration libraries of math routines and deep learning primitives helps Caffe2 users by speeding up training and inference tasks. Vendors of CPUs and GPUs typically offer such libraries, and Caffe2 has support to use such libraries if they are available on your system.

Intel Math Kernel Library (**MKL**) is key to faster training and inference on Intel CPUs. This library is free for personal and community use. It can be downloaded by registering here: `https://software.seek.intel.com/performance-libraries`. Installation involves uncompressing the downloaded package and running the `install.sh` installer script as a superuser. The library files are installed by default to the `/opt/intel` directory. The Caffe2 build step, described in the next section, finds and uses the BLAS and LAPACK routines of MKL automatically, if MKL was installed at the default directory.

CUDA and **CUDA Deep Neural Network** (**cuDNN**) libraries are essential for faster training and inference on NVIDIA GPUs. CUDA is free to download after registering here: `https://developer.nvidia.com/cuda-downloads`. cuDNN can be downloaded from here: `https://developer.nvidia.com/cudnn`. Note that you need to have a modern NVIDIA GPU and an NVIDIA GPU driver already installed. As an alternative to the GPU driver, you could use the driver that is installed along with CUDA. Files of the CUDA and cuDNN libraries are typically installed in the `/usr/local/cuda` directory on Linux. The Caffe2 build step, described in the next section, finds and uses CUDA and cuDNN automatically if installed in the default directory.

Building Caffe2

Using Git, we can clone the Git repository containing Caffe2 source code and all the submodules it requires:

```
$ git clone --recursive https://github.com/pytorch/pytorch.git && cd
pytorch

$ git submodule update --init
```

Notice how the Caffe2 source code now exists in a subdirectory inside the PyTorch source repository. This is because of Facebook's cohabitation plan for these two popular DL frameworks as it endeavors to merge the best features of both frameworks over a period of time.

Caffe2 uses CMake as its build system. CMake enables Caffe2 to be easily built for a wide variety of compilers and operating systems.

To build Caffe2 source code using CMake, we first create a build directory and invoke CMake from within it:

```
$ mkdir build
$ cd build
$ cmake ..
```

CMake checks available compilers, operating systems, libraries, and packages, and figures out which Caffe2 features to enable and compilation options to use. These options can be seen listed in the `CMakeLists.txt` file present at the root directory. Options are listed in the form of `option(USE_FOOBAR "Use Foobar library" OFF)`. You can enable or disable those options by setting them to `ON` or `OFF` in `CMakeLists.txt`.

These options can also be configured when invoking CMake. For example, if your Intel CPU has support for AVX/AVX2/FMA, and you would wish to use those features to speed up Caffe2 operations, then enable the `USE_NATIVE_ARCH` option as follows:

```
$ cmake -DUSE_NATIVE_ARCH=ON ..
```

Installing Caffe2

CMake produces a `Makefile` file at the end. We can build Caffe2 and install it on our system using the following command:

```
$ sudo make install
```

This step involves building a large number of CUDA files, which can be very slow. It is recommended to use the parallel execution feature of `make` to use all the cores of your CPU for a faster build. We can do this by using the following command:

```
$ sudo make -j install
```

Using the `make` install method to build and install makes it difficult to update or uninstall Caffe2 later.
Instead, I prefer to create a Debian package of Caffe2 and install it. That way, I can uninstall or update it conveniently. We can do this using the `checkinstall` tool.

To install `checkinstall`, and to use it to build and install Caffe2, use the following commands:

```
$ sudo apt-get install checkinstall
$ sudo checkinstall --pkgname caffe2
```

This command also produces a Debian `.deb` package file that you can use to install on other computers or share with others. For example, on my computer, this command produced a file named `caffe2_20181207-1_amd64.deb`.

If you need a faster build, use the parallel execution feature of `make` along with `checkinstall`:

```
$ sudo checkinstall --pkgname caffe2 make -j install
```

If you need to uninstall Caffe2 in the future, you can now do that easily using the following command:

```
$ sudo dpkg -r caffe2
```

Testing the Caffe2 Python API

We have now installed Caffe2, but we need to make sure it is correctly installed and that its Python API is working. An easy way to do that is to return to your home directory and check whether the Python API of Caffe2 is imported and can execute correctly. This can be done using the following commands:

```
$ cd ~
$ python -c "from caffe2.python import core"
```